

Mock Paper 2 — Algorithms & Programming (H446/02)

Time allowed: 2 hours 30 minutes · Total: 140 marks · Answer all questions.

Instructions

- Use **OCR Exam Reference Language** or a **high-level language** for code.
- **Show all traces** in full (e.g. variable/stack tables for every step).
- Quality of written communication is assessed in the extended-response question.
- Mark your work against the **mark scheme** at the end of this paper.
- Each part is tagged with its mark total [n] and the dominant assessment objective (A01 / A02 / A03) .

Questions

Question 1 — Computational thinking applied to a scenario [12]

A company is building **RidePool**, a ride-sharing app. The system matches a passenger to a nearby driver, calculates a fare, and tracks the journey in real time. Thousands of passengers and drivers use the system at once.

(a) The lead developer says the system will use **abstraction**. With reference to RidePool, explain the difference between **representational abstraction** and **abstraction by generalisation (categorisation)**, giving one example of each. [4] (AO2)

(b) The matching problem is **decomposed** into smaller sub-problems. Identify **two** sensible sub-problems of the matching process and, for each, state one input and one output. [4] (AO2)

.....

(c) Explain why the developers must consider **concurrency** in RidePool, and describe **one** problem that could occur if two passengers request the same driver at the same instant. [2] (AO2)

.....

.....

.....

(d) RidePool **caches** the result of recently requested fare estimates between common pickup and drop-off locations. State **one** benefit and **one** drawback of this caching, given that road conditions change through the day. [2] (AO2)

.....

.....

.....

Running total: 12

Question 2 — Programming techniques [11]

A programmer is writing utility subroutines.

(a) State **one** difference between a **function** and a **procedure**. [1] (AO1)

.....

.....

.....

(b) Explain the difference between passing a parameter **by value** and **by reference**. [2] (AO1)

.....

.....

.....

(c) Consider the code below, written in OCR Exam Reference Language.

```

x = 5

procedure tweak(byVal a, byRef b)
    a = a + 1
    b = b + 1
endprocedure

y = 10
tweak(x, y)
print(x)
print(y)

```

State exactly what is printed, in order, and explain why, referring to value/reference. [3] (AO2)

(d) Explain what is meant by the **scope** of a variable, and state **one** advantage of using **local** variables over **global** variables. [3] (AO1)

(e) Rewrite the following nested-selection code as a **single** decision using logical operators, with no change in behaviour: [2] (AO2)

```

if age >= 18 then
    if hasLicence == true then
        print("May drive")
    endif
endif

```

Running total: 23



Question 3 — Recursion and the call stack [10]

The recursive function `f` is defined below.

```
function f(n)
  if n <= 1 then
    return 1
  else
    return n * f(n - 2)
  endif
endfunction
```

(a) State the **two** features every recursive subroutine must have to terminate correctly. [2] (AO1)

(b) The call `f(7)` is made. Using a **call-stack** trace, show the chain of calls pushed onto the stack and the value returned as each frame is popped (unwinds). [5] (AO2)

(c) State the **final value** returned by `f(7)`. [1] (AO2)

(d) Explain what would happen if the base case were changed to `if n == 0` and `f(7)` were called, and name the run-time error that results. [2] (AO2)

Question 4 — Object-oriented programming [14]

A library system models loanable items. Complete the class definitions below (OCR Exam Reference Language or a high-level language).

(a) Define a class `Book` with **private** attributes `title`, `author` and `available` (boolean). Write a **constructor** that sets `title` and `author` from parameters and sets `available` to `true`. **[5] (AO3)**

(b) Add two methods to `Book`: - `borrowItem()` — if the book is available, set `available` to `false` and return `true`; otherwise return `false`. - `returnItem()` — set `available` to `true`. **[4] (AO3)**

(c) A class `ReferenceBook` **inherits** from `Book` but may never be borrowed. Write `ReferenceBook` so that it inherits from `Book` and **overrides** `borrowItem()` to always return `false`. **[3] (AO3)**

(d) State which OOP concept is demonstrated by making the attributes private and accessing them only through methods, and give **one** advantage of doing so. **[2] (AO1)**

Running total: 47

Question 5 — Computational methods [12]

(a) A puzzle solver places 8 queens on a chessboard so none attack each other. It tries a placement, and when it reaches a dead end it returns to the most recent choice and tries the next option. Name this computational method and state the data structure conceptually used to remember earlier choices. [2] (AO1)

(b) Explain the difference between an algorithm that always finds the **optimal** solution and a **heuristic** approach. Give **one** reason a heuristic might be preferred for a real-time RidePool driver-matching problem. [3] (AO2)

(c) **Merge sort** is described as a **divide-and-conquer** algorithm. Explain what is meant by *divide and conquer* with reference to how merge sort works. [3] (AO2)

(d) A supermarket analyses millions of loyalty-card transactions to find products frequently bought together. Name this computational method and describe **one** way the results could be **visualised** to support a business decision. [2] (AO2)

(e) State **one** problem for which an exact, optimal solution is **not** computable in a reasonable time, justifying why a heuristic is used instead. [2] (AO2)

Running total: 59

Question 6 — Big O analysis [10]

(a) State what is meant by **Big O notation**, and explain why lower-order terms and constant factors are discarded. [2] (AO1)

(b) State the **worst-case** time complexity, using Big O, of each: - (i) binary search of a sorted list of n items - (ii) bubble sort of n items - (iii) pushing one item onto a stack implemented as an array - (iv) a recursive routine whose number of calls **doubles** for each extra input item [4] (AO1)

(c) Consider the algorithm below.

```
total = 0
for i = 0 to n - 1
  for j = 0 to n - 1
    total = total + arr[i] * arr[j]
  next j
next i
```

State its time complexity in Big O and **justify** your answer. [2] (AO2)

(d) A second algorithm runs in $O(n \log n)$. Explain why, for large n , it is preferable to one running in $O(n^2)$, and state **one** standard sorting algorithm with $O(n \log n)$ average-case complexity. [2] (AO2)

Running total: 69

Question 7 — Searching algorithms [11]

(a) State the **precondition** that must hold for a **binary search** to work correctly. [1] (AO1)

(b) Trace a **binary search** for the value **44** in the sorted list below (indices 0–8). Show `low`, `high`, `mid` and `list[mid]` at each step, and state the number of comparisons of `list[mid]` made. [5] (AO2)

Index:	0	1	2	3	4	5	6	7	8
Value:	3	8	15	22	31	44	58	67	90

(c) Complete the **linear search** function below so that it returns the **index** of `target` if found, or `-1` if not found. [3] (AO3)


```
function linearSearch(list, target)
  for i = 0 to list.length - 1
    // ... complete ...
  next i
  // ... complete ...
endfunction
```

.....

.....

.....

.....

(d) State **one** circumstance in which a **linear** search would be preferred over a binary search, even on data that could be sorted. **[2] (AO2)**

.....

.....

.....

Running total: 80

Question 8 — Sorting algorithms [13]

(a) A list contains `[5, 2, 8, 1, 4]`. Perform an **insertion sort**, writing the **full state of the list after each value has been inserted** into its correct position in the sorted portion. **[4] (AO2)**

.....

.....

.....

.....

.....

.....

(b) State the **worst-case** and **best-case** time complexity of insertion sort, and describe the kind of input that produces the **best** case. **[3] (AO2)**

.....

.....

.....

(c) A list contains [38, 27, 43, 3, 9, 82, 10] . Show how **merge sort** sorts this list. Clearly show the **splitting** (divide) phase and then each **merge** step until the list is fully sorted. [4] (AO3)

(d) Explain **one** reason merge sort has a better worst-case time complexity than bubble sort, and state **one** disadvantage of merge sort compared with an in-place sort. [2] (AO2)

Running total: 93

Question 9 — Data structure algorithms [12]

(a) A **stack** is implemented as an array `stack` with a `top` pointer (initially `top = -1` , meaning empty). Complete the `push` procedure and the `pop` function below, including handling of the **full** and **empty** conditions for a stack of maximum size `MAX` . [5] (AO3)

```
procedure push(value)
    if top == MAX - 1 then
        print("Stack overflow")
    else
        // ... complete ...
    endif
endprocedure

function pop()
    if top == -1 then
        return "Stack underflow"
    else
        // ... complete ...
    endif
endfunction
```

.....

.....

.....

.....

.....

.....

.....

(b) A circular queue is implemented in an array of size 5 with pointers `front` and `rear`. Explain **why** a circular queue is used in preference to a simple **linear** queue for a fixed-size buffer. **[2] (AO2)**

.....

.....

.....

(c) A singly linked list stores the nodes below. `head` points to node at index 1. Each node holds (`data`, `pointer`) where the pointer is the index of the next node, and `pointer = null` marks the end.

Index:	1	2	3	4
Data:	"Beta"	"Delta"	"Alpha"	"Cati"
Pointer:	3	null	4	2

(i) State the order in which the data is read when the list is **traversed** from `head`. **[2] (AO2)**

.....

.....

.....

(ii) A new node `"Echo"` is added at index 5 and inserted so it appears **between** `"Alpha"` and `"Cati"`. State the pointer values that change and their new values. **[3] (AO3)**

.....

.....

.....

.....

.....

Running total: 105

Question 10 — Trees and traversals [11]

The values 50, 30, 70, 20, 40, 60, 80, 35 are inserted, in that order, into an initially empty **binary search tree (BST)**.

(a) Draw the resulting BST. [3] (AO3)

(b) Give the **pre-order**, **in-order** and **post-order** traversal output of your tree. [4] (AO2)

(c) State what is special about the **in-order** traversal of a BST and give one practical use of this property. [2] (AO1)

(d) State the **worst-case** time complexity of searching for a value in a BST and describe the shape of tree that causes this worst case. [2] (AO2)

Running total: 116

Question 11 — Shortest path (Dijkstra / A*) [12]

Consider the weighted, **undirected** graph below.

$$A-B = 2, \quad A-C = 5, \quad B-C = 1, \quad B-D = 7, \quad C-D = 3, \quad C-E = 6, \quad D-E = 1$$

This image shows a full page of blank primary-ruled paper. It features ten sets of horizontal lines across the page. Each set consists of a solid top line, a dashed midline, and a solid bottom line, providing a guide for letter height and placement. The paper is otherwise completely blank, with no text or markings.

RidePool (from Question 1) must store its set of drivers and, for each passenger request, repeatedly find the **nearest available driver** to a pickup location. New drivers come on and off shift continually, and the system handles thousands of requests per second.

Discuss how you would **design** the data storage and **searching** strategy for this problem. Your answer should:

- identify **suitable data structure(s)** for storing drivers and justify the choice;
- describe a **searching/algorithmic approach** for finding the nearest available driver, referring to relevant computational methods and/or graph or search techniques;
- analyse the **time complexity** implications of your approach as the number of drivers grows;
- **evaluate** at least one **trade-off** in your design (e.g. speed vs memory, accuracy vs responsiveness).

The quality of your written communication will be assessed. **[12] (AO3)**

This image shows a full page of primary-ruled paper. It features ten sets of horizontal lines across the page. Each set consists of a solid top line, a dashed middle line, and a solid bottom line, providing a guide for letter height and placement in handwriting practice. The lines are evenly spaced and extend across the entire width of the page.

Running total: 140

